

HITEC University, Taxila



Department of Computer Engineering
7th Semester

EMBEDDED SYSTEMS
COMPLEX ENGINEERING PROBLEM

Submitted By:

SADIA ZULFIQAR 22-CE-039

MUHAMMAD BIN QAMAR 22-CE-019

Submitted To:

Engr. Qasim Javaid

Problem Statement:

Urban traffic congestion leads to increased travel time, fuel consumption, and environmental pollution. Conventional traffic signal systems operate on fixed timing plans, which fail to respond effectively to real-time traffic variations. You are required to develop an ESP32-based adaptive traffic signal control system capable of estimating real-time vehicle density using non-camera sensors (such as IR, ultrasonic, inductive, or magnetic sensors). Based on this estimation, the system must dynamically adjust signal timing to optimize traffic flow. The system must operate reliably under environmental variability (rain, dust and temperature), noisy sensor data, and strict safety constraints. Additionally, it must address multiple stakeholder expectations, including traffic authorities, commuters, emergency services, and pedestrians.

Objectives:

1. Analyze system requirements and identify appropriate sensing and data acquisition techniques for real-time environment monitoring.
2. Design an embedded system architecture by integrating sensing, processing, communication, and actuation components.
3. Develop adaptive decision-making and control strategies based on real-time data to optimize system behavior.

Tasks:

1. Select appropriate non-vision sensors and justify their use.
2. Design block diagrams, flowcharts, and system architecture.
3. Implement decision-making logic on ESP32.

ESP32-BASED ADAPTIVE TRAFFIC SIGNAL CONTROL SYSTEM

Introduction

Urban traffic congestion has become one of the most critical challenges faced by modern cities worldwide. Rapid population growth, unplanned urban expansion, increased vehicle ownership, and limited road infrastructure have significantly increased traffic density at road intersections. As a result, vehicles spend a considerable amount of time waiting at traffic signals, especially during peak hours.

Traditional traffic signal systems operate using **fixed timing plans**, where each signal is assigned a pre-defined green, yellow, and red duration regardless of actual traffic conditions. While such systems are simple to implement, they lack intelligence and adaptability. They cannot respond to dynamic changes in vehicle density, uneven traffic distribution across lanes, or sudden congestion caused by accidents or peak-hour flow.

This rigid control mechanism leads to several serious consequences:

- **Increased Travel Time:** Vehicles are forced to wait even when there is little or no traffic on adjacent lanes.
- **Fuel Wastage:** Prolonged idling at intersections increases fuel consumption.
- **Environmental Pollution:** Higher fuel usage leads to increased carbon emissions and air pollution.
- **Driver Frustration:** Long waiting times create stress and unsafe driving behavior.
- **Delays for Emergency Vehicles:** Ambulances and fire trucks may be stuck in traffic due to fixed signal timings.
- **Reduced Traffic Efficiency:** Poor coordination results in uneven traffic distribution and congestion buildup.

To overcome these limitations, an **adaptive traffic control system** is proposed. The system is designed using the **ESP32 microcontroller**, a powerful and energy-efficient embedded platform capable of handling real-time sensor data and decision-making processes.

Instead of relying on camera-based systems, which require high computational power and are sensitive to lighting conditions, this system uses **non-vision sensors**, specifically:

- **Infrared (IR) Sensors** for vehicle presence detection.
- **Ultrasonic Sensors** for measuring vehicle distance and estimating traffic density.

By continuously monitoring vehicle presence and distance in each lane, the system estimates real-time traffic density and dynamically adjusts the green signal duration accordingly. Lanes with higher congestion are allocated longer green times, while lanes with low traffic are assigned shorter durations. This ensures balanced traffic flow and reduces unnecessary waiting.

The system incorporates:

- **Emergency Vehicle Priority:** A dedicated IR sensor detects emergency vehicles and immediately grants them signal priority.
- **Pedestrian Crossing Mechanism:** A push-button interface allows pedestrians to request safe crossing time, during which both lanes are temporarily stopped.

The proposed adaptive system improves intersection efficiency, enhances road safety, reduces fuel consumption, and contributes to environmental sustainability. It provides a cost-effective, scalable, and reliable solution suitable for smart city traffic management systems.

Problem Statement

Urban traffic congestion leads to increased travel time, fuel consumption, and environmental pollution. Conventional traffic signal systems operate on fixed timing plans, which fail to respond effectively to real-time traffic variations.

The objective is to develop an ESP32-based adaptive traffic signal control system capable of estimating real-time vehicle density using non-camera sensors (such as IR, ultrasonic, inductive, or magnetic sensors). Based on this estimation, the system must dynamically adjust signal timing to optimize traffic flow.

The system must operate reliably under environmental variability (rain, dust, and temperature), noisy sensor data, and strict safety constraints. Additionally, it must address multiple stakeholder expectations, including:

- Traffic authorities
- Commuters
- Emergency services
- Pedestrians

Objectives

1. Analyze system requirements and select appropriate non-vision sensors.
2. Design embedded system architecture integrating sensing, processing, and actuation.
3. Develop adaptive decision-making logic.

4. Implement emergency vehicle detection.
5. Integrate pedestrian crossing functionality.
6. Ensure system stability and safety constraints.

Selection of Non-Vision Sensors & Justification

IR Sensors

Purpose: Detect presence of vehicles.

Why chosen:

- Low cost
- Fast response
- Low power consumption
- Simple digital output
- Easy integration with ESP32

Ultrasonic Sensors

Purpose: Measure vehicle distance to estimate density.

Why chosen:

- Accurate distance measurement
- Works without camera
- Less affected by lighting conditions
- Suitable for outdoor deployment
- Cost-effective compared to radar

Why Not Camera-Based System?

- High processing requirement
- Privacy concerns
- Expensive hardware
- Performance affected by lighting and weather

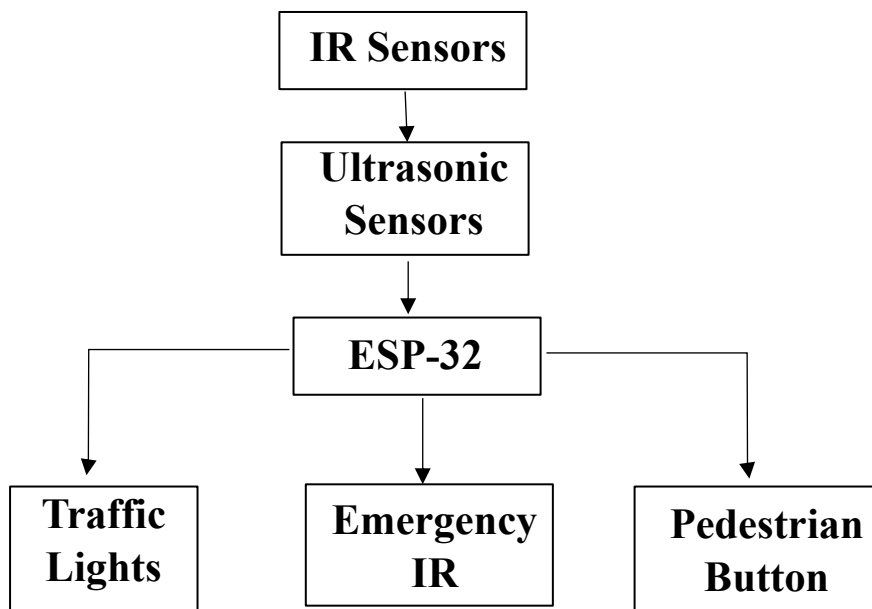
Thus, non-vision sensors were selected for reliability, affordability, and simplicity.

System Architecture

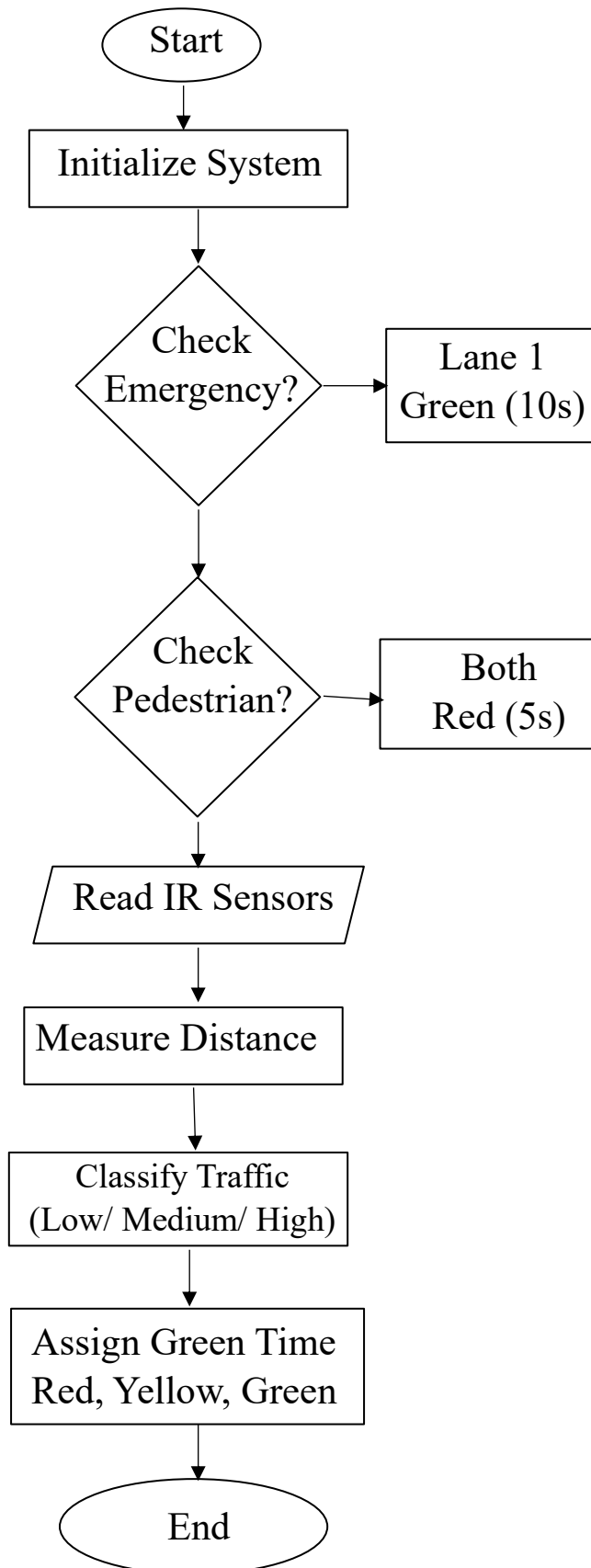
The system consists of:

1. Sensing Layer
 - IR sensors
 - Ultrasonic sensors
 - Emergency IR
 - Pedestrian button
2. Processing Layer
 - ESP32 microcontroller
3. Actuation Layer
 - Traffic light modules
4. Communication Layer
 - Serial monitor for monitoring and debugging

Block Diagram



Flow Chart



How the System Works

Normal Mode

- IR detects vehicle.
- Ultrasonic measures distance.
- Distance < 20cm → HIGH traffic
- 20–40cm → MEDIUM traffic
- 40cm → LOW traffic
- Green time assigned accordingly.

Emergency Mode

- Emergency IR detects ambulance.
- Lane 1 immediately turns GREEN.
- Other lane turns RED.
- After 10 seconds, system returns to adaptive mode.

Pedestrian Mode

- Button pressed.
- Both lanes turn RED.
- 5 seconds crossing time allowed.
- System resumes adaptive control.

Hardware Components

Component	Purpose
ESP32	Main controller
2 Ultrasonic Sensors	Measure vehicle distance
2 IR Sensors	Detect vehicle presence
Emergency IR Sensor	Detect ambulance
Push Button	Pedestrian crossing
Traffic LED Module	Indicate signals

Power Supply	5V / 3.3V
--------------	-----------

Hardware Connections

Ultrasonic Lane 1

- Trig → GPIO 13
- Echo → GPIO 14

Ultrasonic Lane 2

- Trig → GPIO 12
- Echo → GPIO 18

IR Sensors

- IR1 → GPIO 25
- IR2 → GPIO 26

Emergency IR

- GPIO 32

Pedestrian Button

- GPIO 27 → Button → GND (INPUT_PULLUP)

Traffic Lights

Lane 1:

- Red → GPIO 21
- Yellow → GPIO 22
- Green → GPIO 23

Lane 2:

- Red → GPIO 19
- Yellow → GPIO 4
- Green → GPIO 2

Code

```
// ----- PIN DEFINITIONS -----  
  
// Ultrasonic Sensors  
#define trig1 13  
#define echo1 14  
#define trig2 12  
#define echo2 18  
  
// IR Sensors  
#define IR1 25  
#define IR2 26  
  
// Emergency IR (PIN 32)  
#define EMERGENCY_IR 32  
  
// Pedestrian Button (Moved to safe pin)  
#define PED_BUTTON 27  
  
// Traffic Light Lane 1  
#define R1 21  
#define Y1 22  
#define G1 23  
  
// Traffic Light Lane 2  
#define R2 19  
#define Y2 4  
#define G2 2
```

```
unsigned long previousMillis = 0;
unsigned long interval = 0;

int state = 0;
int currentLane = -1;

unsigned long lastPedPress = 0;
const unsigned long debounceDelay = 1000;

void allRed() {
    digitalWrite(R1, HIGH);
    digitalWrite(R2, HIGH);
    digitalWrite(G1, LOW);
    digitalWrite(G2, LOW);
    digitalWrite(Y1, LOW);
    digitalWrite(Y2, LOW);
}

int readUltrasonic(int trig, int echo) {
    digitalWrite(trig, LOW);
    delayMicroseconds(2);
    digitalWrite(trig, HIGH);
    delayMicroseconds(10);
    digitalWrite(trig, LOW);

    long duration = pulseIn(echo, HIGH, 25000);
    int distance = duration * 0.034 / 2;
    return distance;
}
```

```
}

void setup() {

    Serial.begin(115200);

    pinMode(trig1, OUTPUT);
    pinMode(echo1, INPUT);
    pinMode(trig2, OUTPUT);
    pinMode(echo2, INPUT);

    pinMode(IR1, INPUT);
    pinMode(IR2, INPUT);

    pinMode(EMERGENCY_IR, INPUT);
    pinMode(PED_BUTTON, INPUT_PULLUP);

    pinMode(R1, OUTPUT);
    pinMode(Y1, OUTPUT);
    pinMode(G1, OUTPUT);

    pinMode(R2, OUTPUT);
    pinMode(Y2, OUTPUT);
    pinMode(G2, OUTPUT);

    allRed();

    Serial.println("===== 2-WAY ADAPTIVE TRAFFIC SYSTEM =====");
```

```

}

void loop() {

    unsigned long currentMillis = millis();

    // ===== EMERGENCY =====

    if (digitalRead(EMERGENCY_IR) == LOW) {

        Serial.println("🚒 EMERGENCY VEHICLE DETECTED");
        Serial.println("Lane 1 GREEN (Emergency)");
        Serial.println("Lane 2 RED");

        allRed();
        digitalWrite(R1, LOW);
        digitalWrite(G1, HIGH);

        delay(10000);

        digitalWrite(G1, LOW);
        digitalWrite(R1, HIGH);
        return;
    }

    // ===== PEDESTRIAN =====

    if (digitalRead(PED_BUTTON) == LOW) {

        if (currentMillis - lastPedPress > debounceDelay) {

```

```

Serial.println("🚶 Pedestrian Crossing Activated");
Serial.println("Both Lanes RED");

allRed();
delay(5000);

lastPedPress = currentMillis;
}
}

// ===== STATE MACHINE =====
if (state == 1 && currentMillis - previousMillis >= interval) {

    if (currentLane == 1) {
        digitalWrite(G1, LOW);
        digitalWrite(Y1, HIGH);
        Serial.println("Lane 1 YELLOW");
    } else {
        digitalWrite(G2, LOW);
        digitalWrite(Y2, HIGH);
        Serial.println("Lane 2 YELLOW");
    }

    previousMillis = currentMillis;
    interval = 3000;
    state = 2;
}

```

```
else if (state == 2 && currentMillis - previousMillis >= interval) {
```

```
    if (currentLane == 1) {  
        digitalWrite(Y1, LOW);  
        digitalWrite(R1, HIGH);  
        Serial.println("Lane 1 RED");  
    } else {  
        digitalWrite(Y2, LOW);  
        digitalWrite(R2, HIGH);  
        Serial.println("Lane 2 RED");  
    }  
}
```

```
state = 0;  
}
```

```
// ===== ADAPTIVE MODE =====
```

```
if (state == 0) {
```

```
    int selectedLane = -1;  
    int densityScore1 = 0;  
    int densityScore2 = 0;
```

```
    int distance1 = 0;  
    int distance2 = 0;
```

```
    if (digitalRead(IR1) == LOW) {
```

```

distance1 = readUltrasonic(trig1, echo1);

if (distance1 > 5 && distance1 < 20)
    densityScore1 = 3;
else if (distance1 < 40)
    densityScore1 = 2;
else
    densityScore1 = 1;
}

if (digitalRead(IR2) == LOW) {

    distance2 = readUltrasonic(trig2, echo2);

    if (distance2 > 5 && distance2 < 20)
        densityScore2 = 3;
    else if (distance2 < 40)
        densityScore2 = 2;
    else
        densityScore2 = 1;
}

if (densityScore1 > densityScore2)
    selectedLane = 1;
else if (densityScore2 > densityScore1)
    selectedLane = 2;
else
    return;

```

```
int selectedDensity = (selectedLane == 1) ? densityScore1 : densityScore2;
```

```
int greenTime;
```

```
if (selectedDensity == 3) {
```

```
    greenTime = 20000;
```

```
    Serial.println("Traffic Level: HIGH");
```

```
}
```

```
else if (selectedDensity == 2) {
```

```
    greenTime = 12000;
```

```
    Serial.println("Traffic Level: MEDIUM");
```

```
}
```

```
else {
```

```
    greenTime = 6000;
```

```
    Serial.println("Traffic Level: LOW");
```

```
}
```

```
allRed();
```

```
if (selectedLane == 1) {
```

```
    digitalWrite(R1, LOW);
```

```
    digitalWrite(G1, HIGH);
```

```
    Serial.println("Lane 1 GREEN");
```

```
    Serial.println("Lane 2 RED");
```

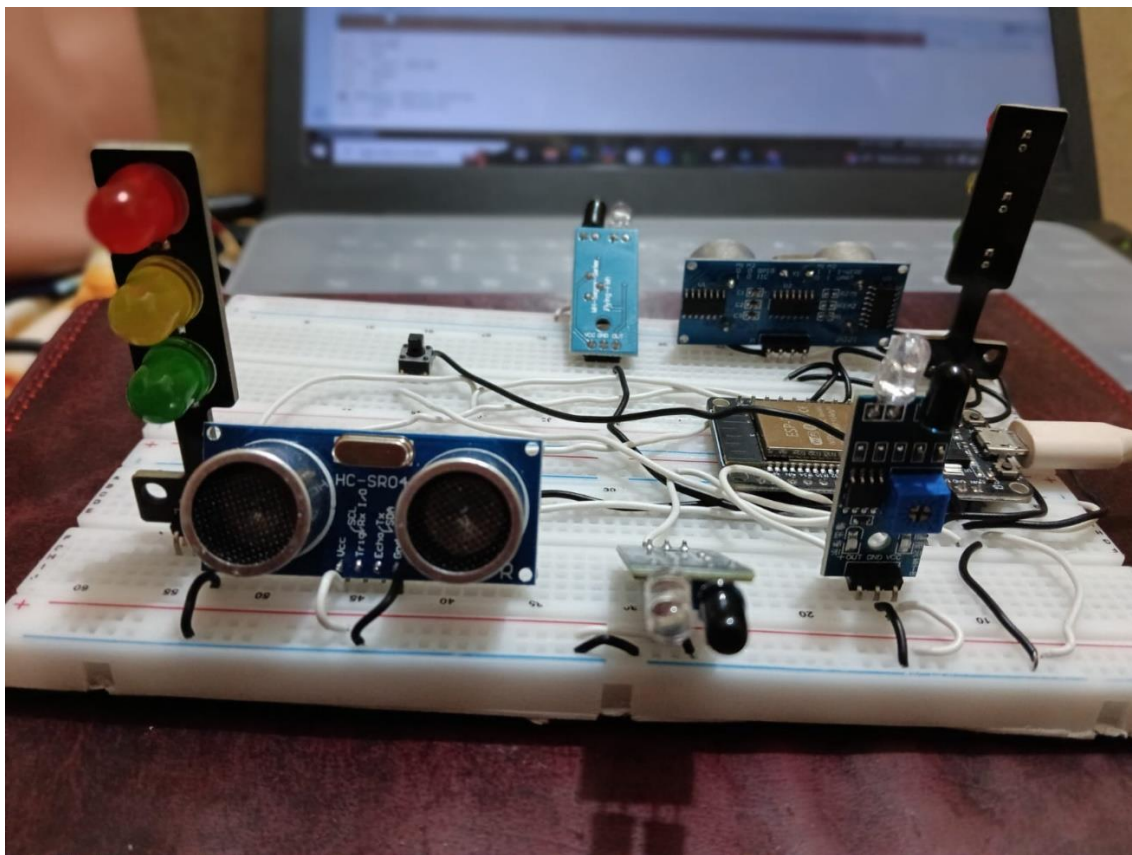
```
} else {
```

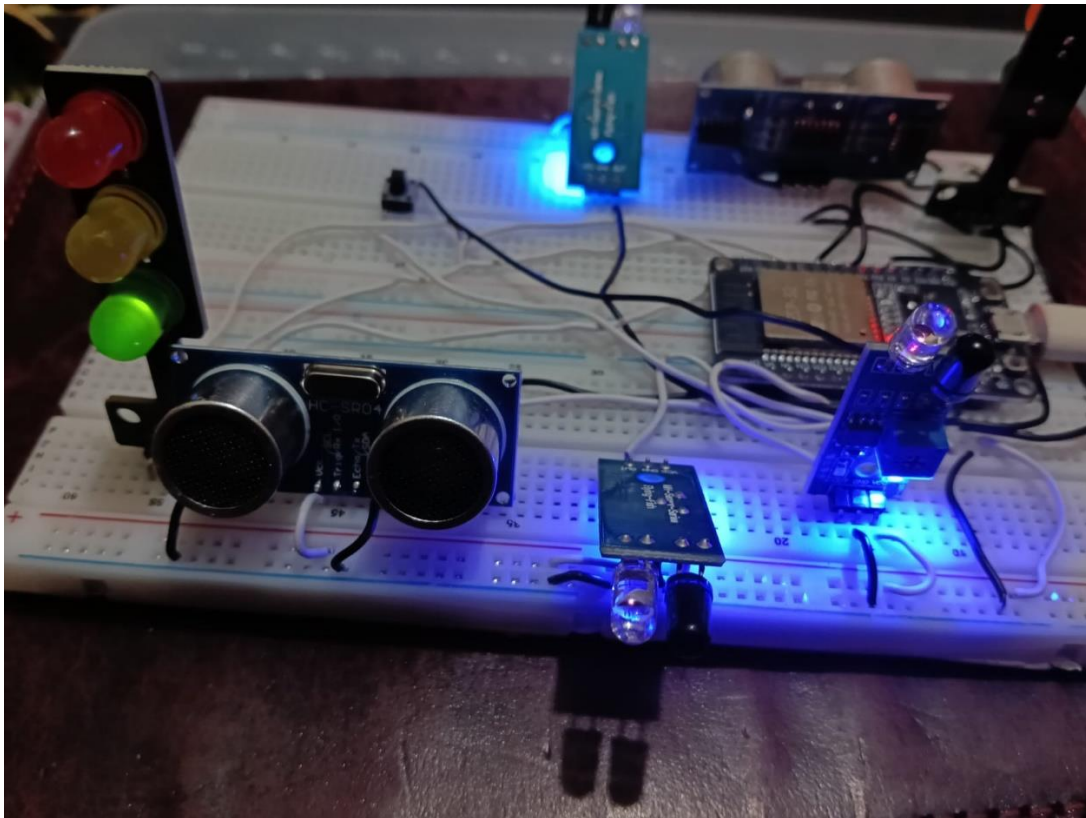
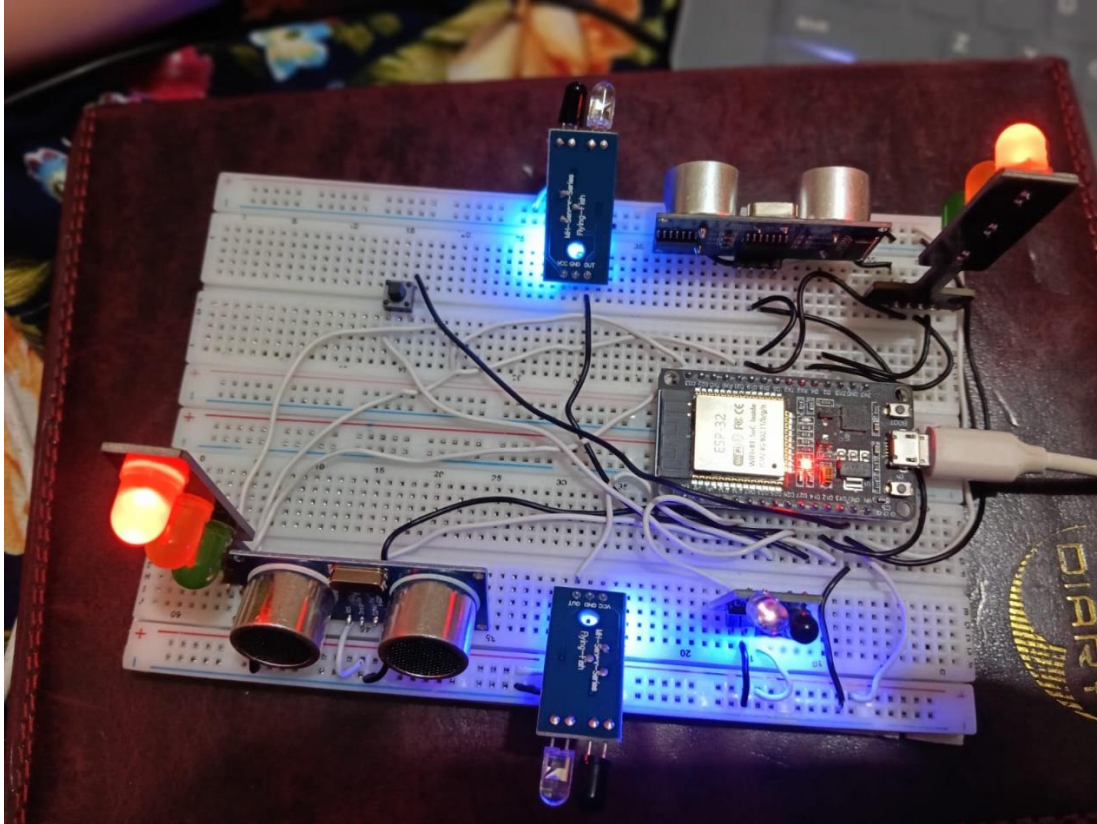
```
    digitalWrite(R2, LOW);
```

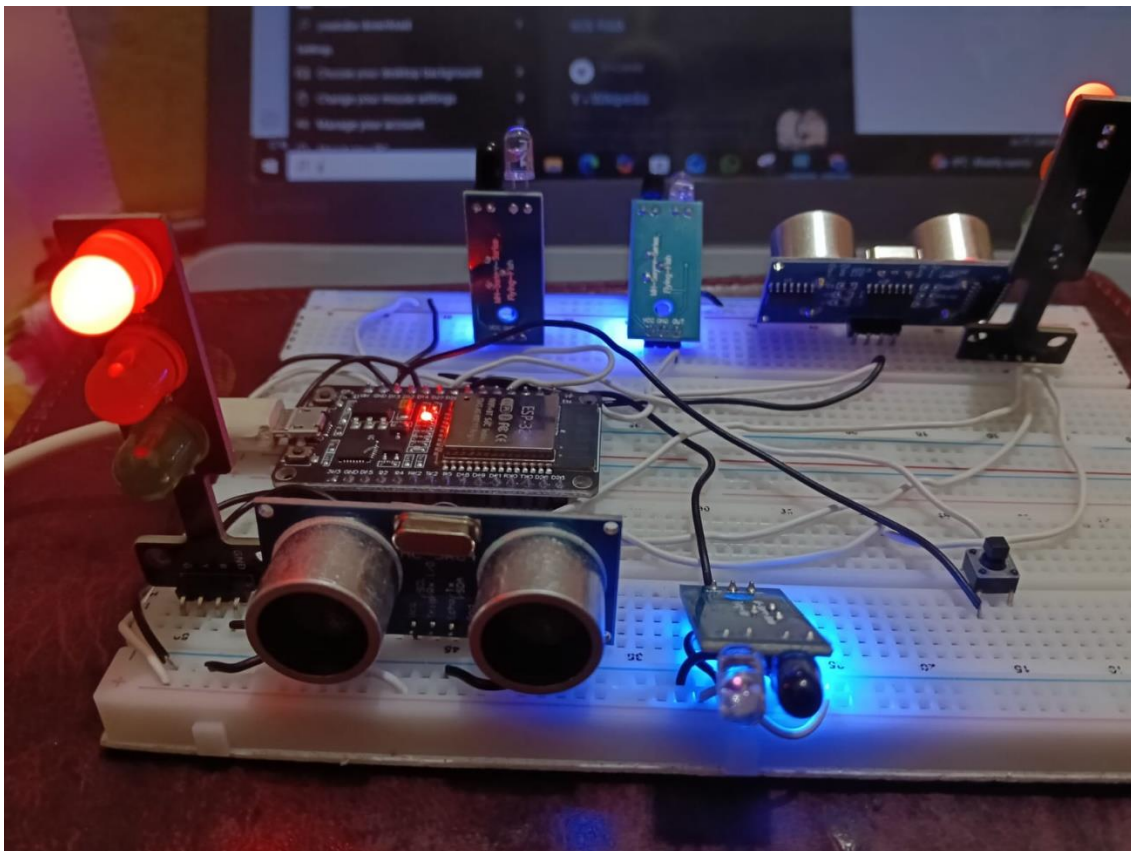
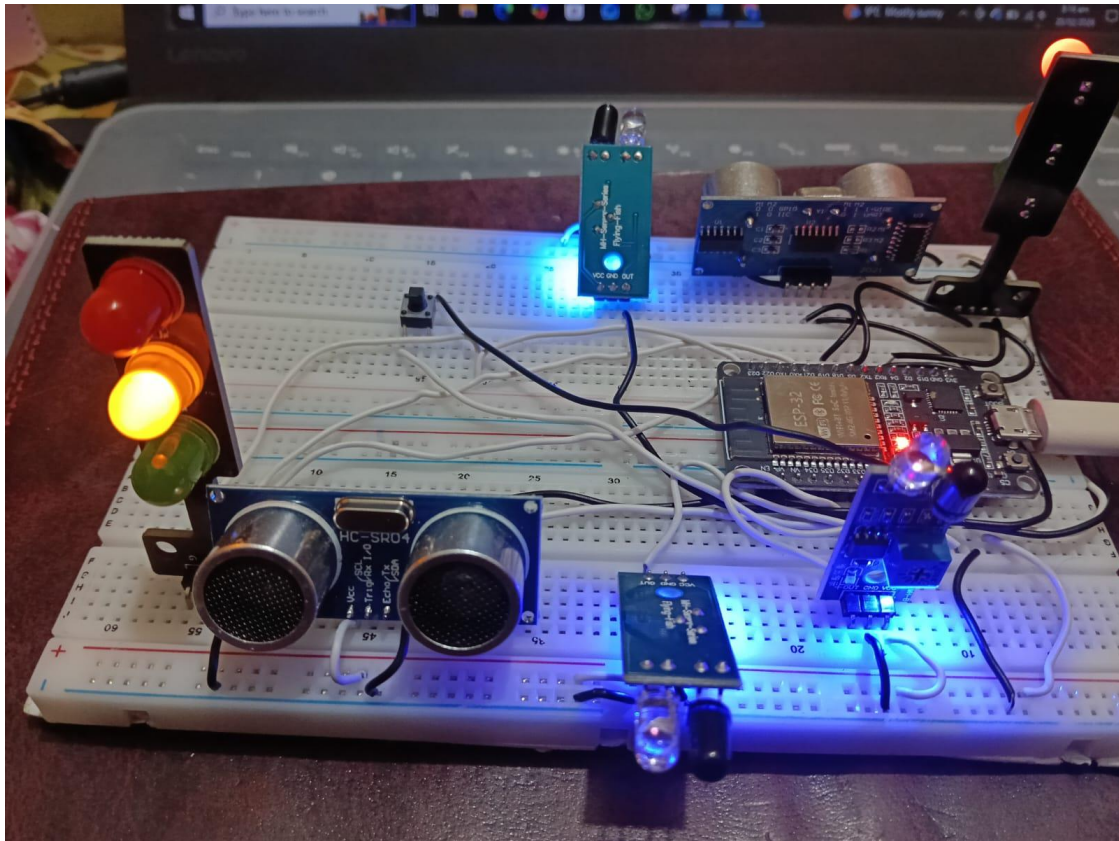
```
    digitalWrite(G2, HIGH);
```

```
Serial.println("Lane 2 GREEN");  
Serial.println("Lane 1 RED");  
}  
  
currentLane = selectedLane;  
previousMillis = currentMillis;  
interval = greenTime;  
state = 1;  
}  
}
```

Output







```

===== 2-WAY ADAPTIVE TRAFFIC SYSTEM =====
🚑 EMERGENCY VEHICLE DETECTED
Lane 1 GREEN (Emergency)
Lane 2 RED
Traffic Level: MEDIUM
Lane 2 GREEN
Lane 1 RED
Lane 2 YELLOW
Lane 2 RED
Traffic Level: MEDIUM
Lane 1 GREEN
Lane 2 RED
Lane 1 YELLOW
Lane 1 RED

🚶 Pedestrian Crossing Activated
Both Lanes RED
Traffic Level: HIGH
Lane 2 GREEN
Lane 1 RED

🚑 EMERGENCY VEHICLE DETECTED
Lane 1 GREEN (Emergency)
Lane 2 RED

🚑 EMERGENCY VEHICLE DETECTED
Lane 1 GREEN (Emergency)
Lane 2 RED
Lane 2 YELLOW

```

The system operates in three main modes: normal adaptive mode, pedestrian mode, and emergency mode. Under normal conditions, the ESP32 continuously monitors both lanes using IR sensors to detect vehicle presence and ultrasonic sensors to measure vehicle distance. Based on the measured distance, traffic density is classified as low, medium, or high, and the lane with higher density is given priority. That lane turns green for a dynamically assigned time, then transitions to yellow, and finally red, while the opposite lane remains red. After completing the cycle, the system again checks which lane has traffic and activates that lane accordingly. When the pedestrian button is pressed, the system immediately overrides normal operation and turns both lanes red for 5 seconds to allow safe crossing, after which adaptive control resumes. In case the emergency IR sensor detects an emergency vehicle, the system gives immediate priority by turning Lane 1 green for 10 seconds while keeping the other lane red; after this clearance period, the system returns to normal adaptive traffic control. This ensures efficient traffic flow, pedestrian safety, and emergency vehicle priority.

Results

During testing:

- Traffic density correctly classified.
- Green time dynamically adjusted:
 - LOW → 6 sec
 - MEDIUM → 12 sec
 - HIGH → 20 sec
- Emergency detection overrides normal traffic.
- Pedestrian crossing works reliably.
- Serial monitor displays traffic level and signal states.

System successfully reduces unnecessary waiting time.

Advantages

- ✓ Adaptive control reduces congestion
- ✓ Fuel saving
- ✓ Reduced pollution
- ✓ Emergency vehicle priority
- ✓ Low cost implementation
- ✓ No camera required
- ✓ Works in moderate environmental conditions

Alternative Methods

Alternative technologies include:

- Camera-based object detection (YOLO, OpenCV)
- Inductive loop detectors
- Magnetic sensors
- Radar sensors
- AI-based smart traffic systems

Why We Chose IR & Ultrasonic Sensors

IR Sensors

- Low cost
- Simple implementation
- Reliable for object presence detection
- Low power consumption

Ultrasonic Sensors

- Accurate distance measurement
- Weather-resistant (better than vision in dust/light variation)
- Easy integration with ESP32

Non-vision sensors were selected because:

- Lower cost
- Easier deployment
- Less computational requirement
- No privacy concerns

Conclusion

The developed ESP32-based adaptive traffic signal control system successfully:

- Adjusts traffic signal timing dynamically.
- Handles emergency vehicle priority.
- Provides pedestrian crossing facility.
- Operates using non-vision sensors.

The system improves traffic efficiency compared to fixed-time traffic signals and provides a scalable solution for smart cities.

Future improvements may include:

- IoT connectivity
- Cloud data logging
- AI-based predictive traffic modeling
- Multi-lane expansion

